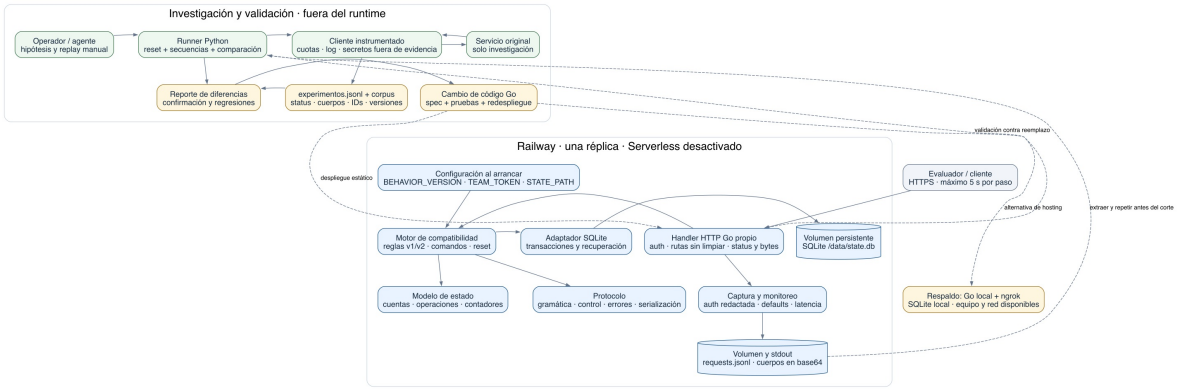
# Caja negra: diseño del reemplazo HTTP

**Estado:** decisiones de diseño ampliadas con evidencia `v1`, 10 de octubre de 2026. Implementación y nuevas confirmaciones pendientes. Los tiempos los controla manualmente el operador. **Objetivo:** reproducir el servicio vigente a la entrega, incluidos errores y memoria entre requests. El reemplazo es autónomo, responde en menos de 5 segundos por paso y conserva su estado tras reinicios. La especificación incremental queda separada de las decisiones de arquitectura; las observaciones de `v1` deben revalidarse tras el cambio.

## Arquitectura y decisiones




Arquitectura del reemplazo y de la investigación

Fuente editable: architecture.mmd. PNG. Coordinación: PROGRESS.md. Contrato: enunciado. Plan experimental: estrategia de descubrimiento.

| Decisión | Razón y límite |
|---|---|
| Go con `net/http`, un proceso y una réplica | Experiencia del equipo, ejecución determinista y operación sencilla. No escalar horizontalmente durante el reto. |
| SQLite en volumen persistente | Un único adaptador para cuentas, operaciones y contadores; bases temporales en pruebas y volumen persistente en entrega. |
| Exploración en Python, fuera del servicio Go | Aprovechar el plan de investigación paralelo. Solo el cliente instrumentado consulta el original; el binario desplegado no contiene ese cliente ni recibe `BB_TOKEN`. |
| Railway; local + ngrok como respaldo | Una réplica, Serverless desactivado y volumen montado. Comprobar cuenta, créditos y continuidad; el respaldo requiere equipo y red disponibles. |
| Actualizaciones mediante código y redespliegue | Detectar cambios con replay manual; no autoadaptación ni consultas al original durante requests del evaluador. |
| Fidelidad antes que mejoras de negocio | Mantener typos, espacios y comportamientos observados. Documentar riesgos sin corregirlos en el reemplazo compatible. |
| Handler propio, sin `http.ServeMux` | Comparar método y ruta sin limpiar ni redirigir: `GET /msg` y `POST //msg` dan 404 E404 (log 41, 44). |
| Reglas versionadas por configuración de arranque | `BEHAVIOR_VERSION=v1/v2`; sin valor, última versión implementada. Versión desconocida impide arrancar. Nunca elegir por reloj. |

El único camino al original es **operador** → **herramienta de exploración** → **cliente instrumentado** → **original**. El evaluador solo llega al **reemplazo** → **SQLite**. La evidencia se convierte en reglas mediante revisión y cambios de código.

## Capas y dependencias

| Capa | Responsabilidad | Dependencias permitidas |
|---|---|---|
| Composición (`cmd/replacement`) | Cargar configuración, abrir almacenamiento, conectar componentes y arrancar/cerrar el servidor. | Adaptador HTTP, motor y almacenamiento concreto. |
| Adaptador HTTP (`internal/httpadapter`) | Capturar método, ruta, autenticación y bytes del cuerpo; escribir status y bytes exactos. Controlar readiness y errores de transporte según evidencia. | Motor y tipos de protocolo. Sin SQL ni reglas de saldos. |
| Motor de compatibilidad (`internal/engine`) | Coordinar validaciones y comandos en el orden observado, seleccionar respuestas, ejecutar consultas/mutaciones/reset de forma indivisible. | Protocolo, modelo de estado y contrato de almacenamiento. Sin HTTP ni cliente del original. |
| Protocolo (`internal/protocol`) | Gramática de `m`, control, representación numérica, errores y serialización compatibles. No normalizar ni usar defaults del framework sin evidencia. | Biblioteca estándar. Sin base de datos ni red. |
| Modelo (`internal/state`) | Tipos de cuentas, operaciones, contadores y cambios de estado; reglas aprendidas, sin inventar restricciones bancarias. | Biblioteca estándar. |
| Persistencia (`internal/storage`) | Implementar el contrato del motor con transacciones y lecturas del estado en SQLite. Persistir resultados completos y recuperar tras reinicio. | Modelo de estado; driver SQLite únicamente en la implementación concreta. |
| Herramientas (`tools/explorer`) | Hipótesis, sondas, secuencias, registro, replay y diferencias. Ejecución manual independiente del runtime. | Cliente HTTP instrumentado y artefactos; nunca importadas por el servicio Go. |

La interfaz de almacenamiento pertenece al consumidor del motor: debe permitir una operación atómica que lea y modifique conjuntamente cuentas, operaciones y contadores. Los adaptadores concretos implementan ese contrato. No dividir una transferencia en escrituras independientes ni fijar todavía la precedencia de validaciones.

## Árbol de directorios propuesto

Los directorios de implementación siguientes son **propuestos**, no código ya creado. El cliente `tools/explorer/bb.py` apareció durante el trabajo paralelo; se reutilizará como única vía al original, verificando sus garantías antes de ampliar el runner.

```
.
├── AGENTS.md
├── PROGRESS.md
├── README.md                        # uso, comandos y enlace al diseño
├── go.mod / go.sum                  # dependencias Go, al implementar
├── cmd/
│   └── replacement/main.go          # composición; único binario desplegado
├── internal/
│   ├── httpadapter/                 # rutas, sobre HTTP y escritura exacta
│   ├── engine/                      # orquestación, comandos y contrato Store
│   ├── protocol/                    # parser, control, errores y respuestas
│   ├── state/                       # cuentas, operaciones y contadores
│   └── storage/
│       └── sqlite/                  # transacciones, esquema y migraciones
├── tools/
│   └── explorer/                    # Python; fuera del despliegue Go
│       ├── cli.py                   # baseline, replay y comparación
│       ├── bb.py                    # cliente existente; log y cuotas
│       ├── sequences.py             # reset y ejecución ordenada
│       ├── probes/                  # hipótesis, fronteras y fuzzing
│       └── compare.py               # semántica JSON y bytes/texto exactos
├── tests/
│   ├── corpus/                      # secuencias confirmadas y versionadas
│   └── differential/                # pruebas con evidencia grabada
├── docs/
│   ├── system-design.md
│   ├── specification.md            # siete secciones, IDs reales y confianza
│   ├── architecture.mmd
│   ├── architecture.svg
│   ├── old-system/                  # enunciado y ejemplos recibidos
│   └── research/                    # estrategia e hipótesis de investigación
├── deploy/
│   └── Dockerfile                   # solo Go; configuración Railway documentada
├── scripts/                         # verificación y generación de entregables
├── experimentos.jsonl              # evidencia registrada; IDs 1–50 en v1
├── artifacts/                      # diferencias y ejecuciones de replay
└── .env                            # local, ignorado; jamás incluido en imágenes
```

SQLite vive en una ruta configurable (`STATE_PATH`) dentro del volumen, por ejemplo `/data/state.db`, no dentro de la imagen ni del repositorio. Archivos auxiliares de SQLite deben quedar en el mismo volumen. `TEAM_TOKEN` se configura en el runtime; `BB_TOKEN` solo en la herramienta de investigación. `.env`, archivos de estado y secretos se excluyen de Git y del contexto de construcción.

Para reducir trabajo, habrá **un solo adaptador de estado: SQLite**, con driver Go puro `modernc.org/sqlite`. Las pruebas usan una base temporal por caso y las mismas transacciones que producción; no se desarrolla un segundo motor en memoria ni snapshots paralelos. Las capas anteriores son límites de responsabilidad, no servicios ni interfaces obligatorias: solo el contrato Store separa motor y persistencia. No añadir repositorios por entidad.

## Estado, atomicidad y operación

Modelo inicial: cuentas (ID, datos de alta, saldo), operaciones (ID, tipo, datos necesarios para reproducir su efecto y anulación) y contadores de IDs. Su estructura definitiva, unidades monetarias, alcance por token y consumo de contadores se determinan con sondas. Saldos, montos y contadores se representan con enteros `int64`, nunca `float64`. Rechazo de decimales, exponentes, overflow y rango son supuestos pendientes de sondas; no convertir ni truncar valores fuera del rango. Serializar enteros sin sufijo `.0`.

Esquema mínimo previsto: `accounts` con ID, datos originales del alta y saldo; `operations` con ID, comando, argumentos originales, deltas por cuenta y marca de anulación; `counters` con el siguiente ID

de cuenta y operación. No imponer restricciones bancarias no observadas. La unidad monetaria y los datos exactos se fijan con evidencia, no por el nombre de los campos. Guardar deltas permite revertir una operación sin recalcular reglas de otra versión.

Una única conexión escritora SQLite y serialización de requests con estado, incluidos consultas y reset, proporcionan una vista consistente. Cada comando que modifica estado confirma una transacción antes de responder; reset borra el alcance y reinicia contadores en una transacción. El alcance exacto del reset sigue pendiente de evidencia. Un error de validación no debe mutar estado salvo que el original demuestre lo contrario.

Supuesto inicial del reemplazo: solo un comando exitoso consume IDs; una anulación ya aplicada no vuelve a mover saldos; reset vacía cuentas y operaciones y vuelve ambos contadores a 1. El único estado de negocio pertenece a `TEAM_TOKEN`. No persistir `X-Req` para compatibilidad, pues los headers no se comparan. Para verificar el supuesto, ejecutar desde reset: alta fallida y exitosa; operación fallida y exitosa; depósito/anulación repetida/consulta; transferencia, anulación y consulta de ambas cuentas; reset y consulta de IDs anteriores, seguido de nuevas altas y operaciones. Comparar también tras reiniciar el reemplazo. Cada divergencia cambia el esquema o la transición y añade una regresión, sin declarar probado el alcance del original.

Parsear importes desde su texto original antes de convertirlos. Sondas obligatorias: `0`, `-1`, `+1`, `01`, espacios, `1.0`, `1e2`, extremos de int64 y valores inmediatamente fuera; probar tanto el importe como el saldo resultante de sumas y transferencias. Las comprobaciones de overflow se hacen antes de escribir. Si el original exige valores fuera de int64, sustituir el tipo por enteros de precisión arbitraria y almacenamiento decimal exacto; no saturar. Mientras falte evidencia, aceptar solo enteros ASCII representables y tratar el rechazo como regla supuesta, con status/cuerpo documentados antes de implementar: no inventar un código nuevo.

No prometer ejecución exactamente una vez ante cortes de red: una operación puede haberse confirmado antes de perder la respuesta. No añadir deduplicación o reintentos de mutaciones que alteren la semántica observada. Readiness exige almacenamiento abierto y esquema listo; la respuesta externa de `/health` debe mantenerse compatible con el contrato observado.

## Contrato HTTP observado y límites de transporte

Usar un `http.Handler` propio como `Server.Handler`, sin mux, redirects ni páginas de error predeterminadas. Las rutas observadas son exactamente `POST /msg`, `POST /__reset` y `GET /health`. Otros métodos y rutas probados devuelven 404 y `{"error":"E404","mensaje":"ruta desconocida"}` (log 41-44). No limpiar `//msg` ni resolver segmentos `..`. La normalización de escapes, HEAD, OPTIONS y requests rechazados por `net/http` antes del handler requiere sondas: no afirmar cobertura.

Health y reset autenticados devuelven 200 `{"ok":true}` (log 34-35); sin token dan 401 `{"error":"E001","mensaje":"no autorizado"}` (log 36, 50). El chequeo de disponibilidad debe enviar `TEAM_TOKEN`; confirmar que el hosting permite ese header antes de configurar un health check. No añadir una ruta pública de readiness. El supuesto para otros tokens es 401 E001, con estado únicamente para `TEAM_TOKEN`. Auth frente a ruta inexistente o JSON inválido sigue sin precedencia demostrada.

Orden provisional, **supuesto** en combinaciones no observadas: autenticación, método/ruta exactos, decodificación JSON, extracción de `m`, gramática del control E201, valor E202, comando/aridad y validaciones de estado. PING conserva sus reglas observadas y se investiga por separado. Las etapas generan una respuesta y terminan; ninguna etapa anterior al comando muta estado. La precedencia entre errores de negocio no se fija sin controles válidos.

Matriz de cierre HTTP, siempre mediante el cliente instrumentado y reset por secuencia:

| Grupo | Sondas | Decisión que debe cerrar |
|---|---|---|
| Precedencia | Ruta inexistente sin token y JSON { sin token. | Auth frente a ruta y JSON. |
| Cuerpo | Vacío, `[]`, `"x"`, `null`, `{"m":""}`, `{"m":null}` y JSON seguido de otro valor. | Forma aceptada y error exacto; no asumir que existe E101. |
| PING | `PING`, `PING;5X`, `PING;64;1`, control vacío y espacios. | Exención de control, aridad y precedencia. |
| Métodos | HEAD, OPTIONS y PUT en `/msg` y `/health`, con y sin auth. | Status y cuerpo efectivo, incluida ausencia de cuerpo en HEAD. |
| Ruta | `//msg`, `/a/../msg`, `/%6dsg`, `/msg/`, `/msg?x=1` y escapes inválidos. | Comparación de ruta original, escapes y query. |
| Transporte | JSON con/sin Content-Type, Expect y bytes no UTF-8; fallos anteriores al handler si el cliente puede emitirlos. | Límites de net/http y bytes preservados. |

Conservar `RequestURI` para captura y comparar la ruta escapada original sin query; nunca basar el routing en una ruta decodificada que convierta `/%6dsg` en `/msg`, salvo evidencia. El cliente debe permitir omitir Content-Type, controlar headers con una lista segura y guardar request/response en base64 además del texto: su decodificación con reemplazo no prueba igualdad de bytes. No enviar sondas manuales fuera del cliente. Si net/http impide reproducir un caso confirmado antes del handler, documentar el límite y adaptar solo ese caso; el handler propio no demuestra cobertura del transporte completo.

## Versionado, captura del evaluador y monitoreo

`BEHAVIOR_VERSION` se lee solo al arrancar, junto con `TEAM_TOKEN` y `STATE_PATH`. Cada versión conserva un conjunto de reglas; `v2` reutiliza `v1` y sobrescribe su delta, con esquema SQLite común. Rollback: configurar `v1` y reiniciar el mismo artefacto sin recompilar; verificar corpus y compatibilidad del estado antes de usarlo. La versión no aparece en rutas ni respuestas. Cada prueba y cada regla cita una `epoch` del explorador.

Capturar cada request del evaluador en `/data/requests.jsonl` y stdout: timestamp UTC, ID local de secuencia, método, ruta y query sin normalización, cuerpo exacto en base64, status, respuesta exacta en base64, duración, versión y regla por defecto utilizada. Un reset autenticado exitoso delimita una secuencia; conservar también los resets fallidos. Omitir `Authorization`, cookies y headers secretos, sin importar sus mayúsculas; registrar solo etiqueta de auth (equipo/otro/ausente) y headers permitidos explícitamente. Si un cuerpo contiene un token configurado, redactarlo antes de cualquier salida y marcar que sus bytes no son reproducibles. Probar ausencia de secretos en JSONL y stdout.

No crear `/logs` ni `/metrics`: acceder por herramientas del hosting. Rotar el archivo a 10 MiB y conservar archivos rotados hasta extraerlos; nunca perder evidencia silenciosamente. El fallo de escritura no cambia status/cuerpo: avisar por stderr y conservar stdout si está disponible. Antes del corte de tokens, extraer secuencias y repetirlas con hipótesis y reset mediante `bb.py`, sin caché, bajo cuota y coordinación exclusiva del token.

Contadores internos: total, status, código de error, defaults, duración máxima y p95 en una ventana de 1.000 requests. Emitir resumen cada 100 requests o minuto y alerta sobre 1 s. Registrar arranque con versión, almacenamiento y esquema listos; nunca secretos. Estas capacidades están diseñadas y siguen pendientes de implementación y verificación.

## Condiciones de despliegue y continuidad

El despliegue se acepta con volumen accesible, una réplica, estado recuperado, recursos disponibles hasta conocer el resultado final y un smoke test público de todas las rutas. Registrar la verificación de créditos y límites de la cuenta, sin afirmar que ya se realizó. El proceso no escucha hasta abrir SQLite y completar el esquema; un fallo impide el arranque.

Primera opción de readiness: `/health` con Authorization, si la plataforma permite configurarlo. Si no lo

permite, usar la comprobación de proceso/TCP disponible y un smoke test externo autenticado; no configurar un chequeo HTTP sin token que recibirá 401 ni abrir otra ruta. Si tampoco hay chequeo alternativo, documentar esa limitación y validar manualmente antes de entregar el endpoint. El respaldo local usa el mismo artefacto y una copia consistente de SQLite, conserva TEAM_TOKEN y se prueba antes de necesitarlo; cambiar el entrypoint solo tras verificarlo. No copiar archivos SQLite abiertos: usar backup SQLite o detener el proceso antes de copiar.

# Investigación y validación

El log 1-50 ya reproduce el corpus y mide precedencia y sobre HTTP en `v1`. Orden observado: E201 antes de E202, y E202 antes de E301/E302/E401 (log 23-32). Priorizar la fórmula del control: descubrir comandos con controles arbitrarios queda bloqueado por E202. Investigar en tandas de hasta 100 solicitudes con hipótesis nuevas. Después cubrir estado, anulaciones, límites, sobre HTTP y secuencias nuevas de 2-12 pasos. La numeración de errores y los nombres bancarios orientan sondas; no prueban reglas ni comandos adicionales.

El cliente exige hipótesis y registra **cada intento**, incluidos reset, health, rutas y autenticación inválida, antes de abrir la conexión. Conserva los campos del README y añade método/ruta, bytes de request y response cuando existe, timestamp, versión de corpus y etiqueta derivada del token sin su valor secreto. El bloqueo del log asigna IDs únicos. Un fallo de transporte permanece como intento sin respuesta y no prueba comportamiento; si ocurre en una secuencia con estado, abortar y repetir desde reset, sin reintento ciego.

El limitador coordina persistentemente entre procesos: 8 req/s, contador por token/día y tope local conservador configurable de 7.800 (nunca por encima del máximo externo de 8.000), incluyendo resets y reintentos. La reserva de turno y presupuesto es atómica en el mismo área privada del explorador; no contiene el token, solo una etiqueta derivada. Las respuestas `X-Frontier-Infra: rate-limit` se marcan como infraestructura, se excluyen de evidencia y comparaciones y consumen presupuesto local. Aplicar backoff; en secuencias de estado, repetir desde reset cuando sea seguro. La caché usa el prefijo completo desde reset, versión y `Content-Type` efectivo; sus aciertos pendientes persisten para que otra ejecución los reenvíe antes del primer fallo de caché. Se evita para confirmaciones y detección de cambios. El log omite autorización y cualquier aparición de secretos.

## Propiedad exclusiva de una secuencia

Implementar un `flock` por etiqueta de token en el coordinador local del explorador, diferente del lock breve de IDs/cuota. Adquirirlo antes del reset real y mantenerlo hasta cerrar o abortar la secuencia, incluidas esperas y confirmaciones sin caché. Todas las llamadas con ese token pasan por el propietario; varios tokens pueden explorar en paralelo. El reset de otro proceso debe quedar bloqueado, aunque no mueva `X-Req`.

Usar un runner/context manager que conserve el descriptor durante toda la secuencia; los comandos CLI independientes necesitan ejecutarse dentro de ese runner, no fingir exclusión con un lock adquirido solo por request. La caída del proceso libera el flock; se abandona la secuencia incierta y se comienza otra desde reset. Registrar propietario, seq e inicio sin secretos. No robar locks por timeout. En varias máquinas, centralizar llamadas por token en el mismo coordinador o asignar tokens distintos: un flock local no coordina hosts.

Aceptar la implementación cuando dos procesos con el mismo token no intercalen reset/msg, el cierre y una caída liberen el lock y un segundo token no se bloquee. Usar continuidad de X-Req solo como alarma adicional: no detecta resets ni debe exigir IDs globales contiguos si se exploran otros tokens.

## Cierre de la fórmula del control

Cada candidato declara qué texto incluye, si cuenta bytes UTF-8 o caracteres, tratamiento de caja/separadores/espacios y reducción al rango 0-99. Obtener controles válidos para pares que cambien una sola propiedad: nombre ASCII/acentuado, longitud, caja, argumento numérico, ceros iniciales y separador. Buscar controles 0-99 solo tras E201, con cuota reservada y reset para evitar que los intentos exitosos contaminen pruebas de estado. No inferir la fórmula solo de controles conocidos.

Aceptar una fórmula tras repetir casos y acertar controles de mensajes nuevos reservados para validación, en varios comandos y con una variante UTF-8; comprobar también que controles alternativos

dan E202 en la misma versión. Vincular algoritmo, fixtures e IDs. Si falla un caso, seguir como hipótesis; cuando el operador cierre la búsqueda, activar explícitamente el plan B y marcar la versión como incompleta. Los valores conocidos solo se reutilizan para el mensaje exacto y la epoch correspondiente, sin extrapolación ni consultas del runtime al original.

## Un comando manual para detectar cambios

Interfaz prevista, **todavía no implementada**:

```
python3 tools/explorer/cli.py replay \
  --target original --corpus tests/corpus \
  --compare artifacts/baseline --out artifacts/current
```

La primera ejecución usa `--out artifacts/baseline` sin `--compare`. El comando valida configuración/cuota, hace reset por secuencia, ejecuta en orden, registra intentos y produce un informe por paso con status, cuerpo y efecto observable. Compara JSON sin convertir números a floats ni ordenar arrays, y texto no JSON de forma exacta; además informa diferencias de bytes para cumplir el requisito más estricto de AGENTS.md. No modifica automáticamente la referencia, la especificación ni el servicio. Salida prevista: 0 sin diferencias, 1 con diferencias, 2 ejecución incompleta/error; una ejecución incompleta nunca cuenta como éxito.

Tras un delta: repetir para descartar ruido, diseñar sondas mínimas, citar IDs nuevos, añadir regresiones, cambiar Go y redesplegar conservando el volumen. Validar el corpus anterior y el nuevo; marcar qué reglas fueron reemplazadas. No seleccionar una versión por hora en runtime: configurar la versión vigente a la entrega. Cada fila de la especificación se revalida con IDs nuevos de la `epoch` vigente o se marca histórica/reemplazada; no elevar observaciones de una sola corrida a `confirmado`. El replay incluye auth, rutas y sobre JSON.

Mantener un manifiesto por regla: ID, epoch, secuencia reproducible, IDs de evidencia, status/cuerpo esperado y efecto comprobable mediante consultas posteriores. El corpus cubre cada regla vigente, combinaciones de errores y secuencias de 2-12 pasos; incluir casos capturados del evaluador sin reemplazar las sondas independientes. La repetición confirmatoria usa otro reset y no caché. Si una regla no puede revalidarse, conservarla como supuesto o histórica, sin presentar evidencia anterior como confirmación de la versión actual.

El informe final distingue coincidencias, deltas y casos no ejecutados. Solo promocionar una epoch cuando el replay esté completo, el delta tenga evidencia repetida y el reemplazo pase el corpus de esa versión. Las pruebas de versiones anteriores siguen disponibles para rollback, pero no obligan a conservar comportamientos que el original cambió. Archivar manifiesto, informe, log y commit del artefacto juntos para poder reproducir la entrega.

## Horarios de corte y plan B

Los tiempos y la detección del cambio los controla manualmente el usuario. Por su indicación, la confirmación de zona horaria deja de ser una tarea pendiente del diseño. La tabla siguiente queda solo como referencia previa; el operador decide cuándo ejecutar replays, cortar la búsqueda, desplegar y entregar. No se crean automatizaciones ni se elige el comportamiento por reloj.

| Hito del 10 de octubre | Bogotá (UTC-5) | CDMX (UTC-6) | UTC |
|---|---|---|---|
| Replay tras posible cambio en Bogotá | 15:35 | 14:35 | 20:35 |
| Replay tras posible cambio en CDMX | 16:35 | 15:35 | 21:35 |
| Corte de búsqueda de fórmula; activar plan B si no hay fórmula confirmada | 18:00 | 17:00 | 23:00 |
| Replay final completo y casos capturados del evaluador | 18:30 | 17:30 | 23:30 |
| Último despliegue previsto | 19:15 | 18:15 | 00:15 del 11 de octubre |
| Congelar código/configuración y validar endpoint | 19:30 | 18:30 | 00:30 del 11 de octubre |
| Entrega conservadora | 20:00 | 19:00 | 01:00 del 11 de octubre |

Plan B del control, **supuesto**, no regla del original: usar valores conocidos para mensajes ya

registrados; para mensajes nuevos aceptar un campo final de 1-2 dígitos ASCII, 0-99; el resto da E201. No inventar E202 para mensajes nuevos sin fórmula. Mantener separado el comportamiento observado de PING. Este fallback pierde casos de control inválido y puede alterar estado en secuencias: activar solo si falla la investigación y registrar sus usos. Incluir sondas de UTF-8 (bytes frente a caracteres), caja, separadores y ceros iniciales.

Antes de congelar: comprobar créditos, volumen, una réplica y Serverless desactivado, guardar el entrypoint y resultado de la validación; confirmar si el evaluador limita validaciones. Mantener el mismo servicio disponible al menos hasta el día siguiente y hasta conocer el resultado final. La hora exacta de evaluación posterior y los créditos siguen sin confirmar. No redesplegar durante una validación; después del congelado, solo intervenir ante una avería comprobada, registrar motivo y repetir smoke test y validación pública.

# Especificación y entrega de evidencia

La especificación contiene las siete secciones del contrato, con IDs reales del log y confianza por comportamiento. El diseño no es evidencia de implementación.

El log de auditoría conserva todos los intentos y sus IDs originales, sin editarlo ni renumerarlo. La entrega incluye ese `experimentos.jsonl` completo; infraestructura e intentos sin respuesta se identifican mediante `outcome` y no prueban comportamiento. Así no se pierde una solicitud al original ni se rompe la relación ID/número de línea. No sustituirlo por un archivo filtrado.

Exportar adicionalmente `evidencia.jsonl` con entradas `completed`, status/respuesta y sin infraestructura; conserva IDs originales y es un índice auxiliar, no el log canónico. Añadir `intentos-no-validos.jsonl` con la razón de exclusión y un manifiesto con conteos y hashes. Si los organizadores piden otra presentación, generarla como derivado sin alterar la auditoría.

El validador debe verificar JSON por línea, IDs únicos e iguales a posición en el log canónico, hipótesis no vacía, seq y campos obligatorios en respuestas completas. Los intentos inciertos conservan nulls y la explicación; nunca se fabrican status, response o X-Req. Verificar que cada cita de la especificación apunta a una respuesta completa, sin infraestructura, de la epoch declarada y con secuencia reproducible iniciada por reset. Una secuencia abortada no se usa para inferir efectos posteriores; sus respuestas válidas aisladas conservan su contexto. Revisar ausencia de secretos y correspondencia entre bytes capturados y cuerpo antes de exportar. El exportador debe fallar ante una cita inválida, sin generar una entrega aparentemente exitosa.

## Prioridad de cierre

| Prioridad | Trabajo | Condición de aceptación |
|---|---|---|
| P0 | Control y transiciones de estado. | Fórmula confirmada o plan B explícito; fixtures de IDs, anulaciones, errores y reset. |
| P0 | Revalidación de versión vigente. | Manifiesto y replay completos con evidencia nueva; sin reglas históricas presentadas como actuales. |
| P1 | Exclusión por token y contrato HTTP. | Sin intercalado de secuencias; matriz HTTP resuelta o supuestos exactos registrados. |
| P1 | Números y transporte. | Fronteras medidas, sin pérdida numérica ni de bytes; límites del servidor identificados. |
| P1 | Despliegue y entrega. | Smoke test público, recuperación, continuidad verificada y auditoría/citas válidas. |
| P2 | Simplificación. | Solo SQLite y un contrato Store; sin segundo almacenamiento ni interfaces innecesarias. |

Estas decisiones cierran las propuestas de diseño, no sustituyen implementación, solicitudes instrumentadas ni verificación de la cuenta de hosting. No se han obtenido nuevas respuestas del original durante esta revisión.

## Plan de implementación y aceptación

1. **Evidencia:** cliente instrumentado y runner; replay de referencia, auth/rutas/reset y precedencia. Cada comportamiento futuro cita IDs del log, sin tokens.
2. **Compatibilidad:** control y gramática, comandos y estado; conservar casos límite y no asumir monotonía de límites antes de aplicar búsqueda binaria.
3. **Persistencia:** SQLite y reset atómico; verificar que reiniciar conserva saldos, IDs y anulaciones, y que un reset restaura exactamente el estado observado.
4. **Regresión y cambio:** secuencias nuevas, errores combinados y replay manual del delta; pruebas locales del corpus tras cada cambio, sin usar el original en el runtime.
5. **Entrega:** Railway con volumen, smoke test público de auth/health/reset/msg/rutas desconocidas, latencia por paso menor de 5 s, corpus vigente y revisión del log/spec.

El diseño inicial no equivale a compatibilidad demostrada. La entrega final exige evidencia para la especificación, pruebas de persistencia y replay público. La evaluación contiene 100 tests (40 individuales, 45 secuencias de 2-5 pasos y 15 de 6-12); reproducir solo los 20 ejemplos no garantiza puntaje alto.

Fuentes de hosting: Railway trial, planes y límites, Serverless y ngrok Free.

# Caja negra: especificación incremental

**Versión observada:** `v1`. Evidencia: `experimentos.jsonl`, IDs 1-50, tres secuencias con reset. El corpus entregado se reprodujo una vez; se conserva confianza **probable** hasta una confirmación independiente. Este documento no afirma vigencia después del cambio. No hay delta observado aún.

Cada fila observada cita IDs reales del log. Las decisiones del reemplazo llevan **supuesto** y no se presentan como evidencia del original. Tras cada cambio, repetir todas las reglas que se entreguen como vigentes y anotar IDs y versión nuevos; retirar de esa entrega las filas históricas que no se revaliden.

## 1. Entrada

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| ENT-01 | `POST /msg` requiere JSON con `m` string. `{}` y `m` numérico dan E102; campos extra y Content-Type text/plain se aceptan en PING. | log 2-21, 46-49 | probable |
| ENT-02 | Sin auth, auth vacía, token inválido o esquema Basic: 401 `{"error":"E001","mensaje":"no autorizado"}`. Health y reset también requieren auth. | log 36-40, 50 | probable |
| ENT-04 | Token distinto de TEAM_TOKEN: el reemplazo responderá 401 E001. No se conoce el conjunto de tokens válidos del original. | Decisión; sin prueba | supuesto |
| ENT-03 | GET /health y POST /__reset autenticados: 200 `{"ok":true}`. GET /msg, rutas inexistentes y POST //msg: 404 E404, sin redirect. HEAD, OPTIONS y precedencia auth/ruta pendientes. | log 34-35, 41-44 | probable |

## 2. Integridad

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| INT-01 | `ALTA;Lucia;83` da E202 y `ALTA;Lucia;82` crea cuenta. Fórmula y cobertura del control desconocidas. | log 3-4 | probable |
| INT-02 | Control 5X en el depósito observado da E201. Rango, espacios y representación pendientes. | log 18 | probable |
| INT-04 | Sin fórmula al corte: validar valores conocidos; para mensajes nuevos aceptar control ASCII de 1-2 dígitos 0-99, resto E201. Se omite E202 desconocido; puede alterar estado. No describe al original. | Plan B del diseño; sin prueba | supuesto |
| INT-03 | `ALTA;Lucia;Gomez;11` da E302. E202 precede E302 para controles inválidos probados (log 26-28). | log 21 | probable |

## 3. Comandos

Cada fila es el bloque inicial del comando; ampliarlo con argumentos, límites, efectos y errores una vez medidos. No limitar la exploración a estos seis comandos.

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| CMD-PING | `PING;64` y `PING;1` → 200, `{"pong":true}`; no basta para afirmar que PING ignora cualquier control. | log 2, 33 | probable |
| CMD-ALTA | Alta de Lucia/Mateo → 201, `{"cuentaId":"AC-0001"}` / `AC-0002`; las cuentas observadas arrancan con saldo 0. | log 4-6 | probable |
| CMD-CONSULTA | Consulta devuelve 200 y claves `cuenta_id`, `saldo`; `CONSULTa` funciona en ese caso. Normalización general pendiente. | log 6, 9, 12, 20 | probable |
| CMD-DEPOSITO | Depósitos observados afectan saldo y devuelven 200, `{"status":"recieved","operacionId":"OP-NNNN"}`. | log 7, 19-20 | probable |
| CMD-TRANSFER | Transferencias observadas mueven saldo y devuelven la misma forma de operación; límites no generalizados. | log 8-12 | probable |
| CMD-ANULA | Primera anulación: 200, `{"ok":true,"anulada":"OP-0003"}`. Repetición: 200, `{"ok":false,"motivo":"ya anulada"}`. Reversión y otros tipos pendientes. | log 14-15, 17, 20 | probable |

## 4. Memoria entre requests

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| MEM-01 | Saldos, cuentas y operaciones afectan requests posteriores; IDs observados consecutivos. Consumo de IDs ante errores pendiente. | log 4-20 | probable |
| MEM-02 | Reset inicia las secuencias observadas; X-Req no se reinicia. Alcance completo e aislamiento por token pendientes. | log 1, 22, 34; contrato | supuesto |
| MEM-03 | Persistir estado tras reinicio es una decisión del reemplazo, no una observación del ciclo de vida del original. | Decisión Q5/Q8; prueba de recuperación pendiente | supuesto |

## 5. Errores

Cuerpos exactos observados; los espacios dentro de strings son significativos. Orden medido: E201 antes de E202, y E202 antes de E301/E302/E401 (log 23-32). El orden entre errores de comando/estado y auth/ruta sigue pendiente.

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| ERR-001 | 401, `{"error":"E001","mensaje":"no autorizado"}`. | log 36-40, 50 | probable |
| ERR-100 | 400, `{"error":"E100","mensaje":"json inválido"}` para cuerpo `{`. | log 45 | probable |
| ERR-102 | 400, `{"error":"E102","mensaje":"falta m"}` para `{}` y `{"m":64}`. | log 46-47 | probable |
| ERR-404 | 404, `{"error":"E404","mensaje":"ruta desconocida"}` para rutas/métodos probados. | log 41-44 | probable |
| ERR-201 | 400, `{"error":"E201","mensaje":"control mal formado"}`. | log 18 | probable |
| ERR-202 | 400, `{"error":"E202","mensaje":"control inválido"}`. | log 3 | probable |
| ERR-301 | 400, `{"error":"E301","mensaje":"comando desconocido "}`; espacio final. | log 13 | probable |
| ERR-302 | 400, `{"error":"E302","mensaje":"aridad inválida"}`. | log 21 | probable |
| ERR-401 | 404, `{"error":"E401","mensaje":"cuenta inexistente"}`. | log 16 | probable |

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| ERR-506 | 422, `{"error":"E506","mensaje":"límite excedido"}` al transferir 120000 en esa secuencia. | log 10 | probable |
| ERR-507 | 422, `{"error":"E507","mensaje":"saldo insuficiente"}` al transferir 50000 tras anulación. | log 17 | probable |

## 6. Cambios durante el día

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| CAM-01 | Se anuncia un cambio a las 15:30; la evaluación usa la versión vigente a las 20:00. No hay delta observado aún. | AGENTS.md y contrato; replay pendiente | supuesto |
| CAM-02 | Anotar versión/corpus, primer timestamp observado y IDs antes/después; sin sondeo continuo no afirmar el instante exacto del cambio. | Decisión Q6/Q10; informe pendiente | supuesto |

## 7. Riesgos

| ID | Comportamiento | Evidencia | Confianza |
|---|---|---|---|
| RIE-01 | La anulación observada no equivale a un diseño bancario seguro; investigar negativos, duplicados y operaciones inexistentes antes de describirlos. Un sistema nuevo requeriría reglas explícitas y auditoría. | log 14-17; sondas pendientes | probable |
| RIE-02 | No cambiar mensajes inconsistentes (`recieved`, claves heterogéneas, espacios) durante el reto. Un cliente nuevo usaría un contrato versionado. | log 4, 6-8, 13 | probable |
| RIE-03 | Reinicios, volumen mal montado, cuota agotada o pérdida de red pueden hacer fallar evaluación. Verificar recuperación, persistencia y endpoint público antes de entregar. | Decisiones Q5/Q9; pruebas pendientes | supuesto |
| RIE-04 | Un fallback desconocido puede reducir puntaje. No inventar comportamiento silenciosamente: medirlo o documentar explícitamente status/ cuerpo y confianza antes de implementar. | Regla del contrato; fallback explícito en INT-04, sin implementación | supuesto |